A project report on

INSTANTB2C – WEB PLATFORM FOR MICRO AND HOME-BASED B2C SELLERS

Submitted in partial fulfillment for the award of the degree of

M.tech Integrated Computer Science and Engineering with Collaboration with Virtusa

by

22MIC7072-SALE MANIDEEP

22MIC7096-CHINTALA SRI CHARAN

22MIC7038- INDURTHI HEMASRI

VIT-AP UNIVERSITY

AMARAVATI

Name of the school

SCOPE

August, 2025

CERTIFICATE

This is to certify that the thesis entitled "InstantB2C – Web Platform for Micro and Home-based B2C Sellers" submitted by Sale Manideep (22MIC7072), Chintala Sri Charan(22MIC7096) and Indruthi Hemasri(22MIC7038) from School of SCOPE VIT-AP, for the award of the Summer Internship for the bonafide work carried out by them under my supervision.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The Project report fulfils the requirements and regulations of VIT-AP and in my opinion meets the necessary standards for submission.

Signature of the Guide

ABSTRACT

The growth of the internet has changed the way people do business and makes it easier for software to be delivered as one specific service. An example of this change is the emergence of multi-tenant SaaS applications, which allow diverse sellers to create and manage their online stores in one ecosystem. This system helps the sellers in that it allows customization of storefronts through banners, backgrounds and brand specific themes which aids in reflection of their brand identity. Through this ecosystem, sellers are allowed to upload, organize, and market any products of their choosing and interact with customers through an easy shopping experience.

The whole design is centred around scalability and security, carefully using modular development and database isolation techniques to guarantee strong multi-tenancy. Sellers enjoy adequate resource allocation, reduced costs, strong data protection and efficient resource utilization. At the same time, the shared platform seamlessly scales with the addition of new tenants. The application leverages cutting-edge artificial intelligence, data analytics and Integrated Model Context Protocol to deliver sophisticated and powerful solutions for the digital commerce ecosystem.

This project illustrates the efficiency of e-business solutions by not only equipping sellers with sophisticated tools to grow their online business but also enabling smart features that improve their competitiveness in the digital marketplace. As the internet becomes more prevalent with mobile computing, wireless access and new communication technologies, such SaaS platforms will be instrumental in the development of the electronic business landscape.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to D r . Ngangbam Indrason, Assistant Professor Sr. Grade-1 of School of Computer Science and Engineering (SCOPE), for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in our endeavor. Our association with him is not confined to academics only, but it is a great opportunity on our part of work with an intellectual and expert in the field of Artificial Intelligence and Machine Learning. I would like to express my gratitude to Dr. G. Viswanathan, Dr. S. V. Kota Reddy and to Dr. Nagaraju Devarakonda and SCOPE School, for providing with an environment to work in and for his inspiration during the tenure of the course. In jubilant mood I express ingeniously my whole-hearted thanks Program Chair. all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Amaravati

Date: 04-09-2025

Sale Manideep

Chintala Sri Charan

Indruthi Hemasri

Table of Contents

ABSTRACT 1

ACKNOWLEDGEMENT 2

LIST OF FIGURES 4

LIST OF TABLES 5

LIST OF ACRONYMS 6

CHAPTER 1 7

1.1 INTRODUCTION 7

1.2 OVERVIEW OF MULTI-TENANT SAAS APPLICATION 7

1.3 CHALLENGES PRESENT IN LEGACY SYSTEMS 8

1.4 PROJECT STATEMENT 9

1.5 OBJECTIVES 10

1.6 SCOPE OF THE PROJECT 11-12

CHAPTER 2 13

2.1 INTEGRATING MCP IN MULTI-TENANT SAAS FOR DIGITAL COMMERCE 13

2.2 SURVEY ON MULTI-TENANT SAAS AND LEGACY SYSTEMS 17

CHAPTER 3 18

3.1 SYSTEM ARCHITECTURE AND TECHNOLOGY STACK 18-19

3.2 FRONTEND IMPLEMENTATION 20

3.3 BACKEND IMPLEMENTATION 23

3.4 MODEL CONTEXT PROTOCOL INTEGRATION 26

CHAPTER 4 ..... Error! Bookmark not defined.

4.1 TEST INFO 29

4.2 TEST CASES 29

4.3 TEST SUMMARY 34

CHAPTER 5 ..... Error! Bookmark not defined.

5.1 CONCLUSION ..... **Error! Bookmark not defined.**

REFERENCES ..... Error! Bookmark not defined.

LIST OF FIGURES

1st Dataflow Diagram

- Frontend UI Components ……12
- Claude LLM and Database Interaction ……12

2nd Dataflow Diagram

- Communication Frontend UI with Backend server and Database …14
- MCP Server on Desktop Communicating with Database ……14

3rd Dataflow Diagram

Communication Frontend UI with Backend server, Database and LLM .21

Component Diagram …….19

Activity Diagram ……21

Class Diagram ……25

Sequence Diagrams

- Sequence Diagram for placing an order …….23
- Sequence diagram of adding a product ……24
- Sequence diagram of adding a category …….24
- Sequence diagram of user using MCP …….25

LIST OF TABLES

Single-Tenant SaaS vs Multi-Tenant-SaaS……8

Challenges Of Legacy E-Commerce Systems……9

Technology Stack……18

MCP Server Methods Table…….27

Table 4.1: Test Cases for User Authentication ……29

Table 4.2 details the test cases for the product management functionality ……30

Table 4.3: AI Analytics Module Test Cases ……33

LIST OF ACRONYMS

MCP - Model Context Protocol

B2C - Business-to-Consumer

JWT - JSON Web Token

API -Application Programming Interface

ORM - Object-Relational Mapping

SaaS - Software as a Service

LLM - Large Language Model

AI - Artificial Intelligence

CHAPTER 1

1.1 INTRODUCTION

With each passing year, e-commerce keeps expanding and gives a chance to new businesses to establish a global outreach. There more than one reason why custom independent systems seem like a bad investment, they do complicated maintenance and costly too. Solving these problems offers numerous benefits. Over the years the use of these modular constructs has increased, which has led to the conceptualization of Software as a Service (SaaS) applications. These will help automobile sellers as they will no longer be siloed and will use the same norms. These businesses will scale and be independent from each other along with the IT and Telecomm benefits.

This project centers around the development of a multi-tenant SaaS application enabling numerous sellers to independently set up their online stores in a single ecosystem. Every seller gets the opportunity to personalize their store by adding unique banners, themes, and backgrounds to carve out their own identity. Sellers are able to set their own product listings, control their own inventory, actively sell their products, and even buy products from other sellers within the marketplace. One of the standout attributes of this project is the AI-powered assistant which interfaces with sellers and offers predictive insights based on their historical sales. This assistant is implemented using the Model Context Protocol (MCP), which guarantees purposeful and rational engagement. Thus, the system integrates marketplace with AI to provide a unique answer to evolving e-commerce problems.

1.2 OVERVIEW OF MULTI-TENANT SAAS APPLICATION

A multi-tenant SaaS application shows a forward-thinking take on software architecture by using one platform to serve several different sellers at the same time, while also keeping their data and activities separate. Each seller, despite using the same infrastructure, gets the impression that they are using a standalone store. This impression is different from single-tenant setups where separate infrastructure is allocated to every individual seller. The multi-tenant model uses shared resources. This model also enhances manageability which in turn, reduces operational costs, improves scalability and reduces operational costs. Reliability in user experience is a consequence of central server management for tasks such as updates, system security, and system maintenance.

The every seller in the proposed system is allocated a customizable storefront which is not a rigid design as they can upload products, design layouts and organize categories to correlate with their business identity. This stripe of distinction makes the seller able to customize their brand with several business design options such as banners, and background themes. Each store is unique in a way that sells the sellers vision.

The most unique element of the system is the Voice of the Business Assistant which is integrated using the Model Context Protocol (MCP) and deployed via the programmable assistant feature. This kinds of assistant is far from a simple chatbot. By interrogating historical sales data, finding behavioral patterns and forecasting demand, this assistant provides sellers with predictive insights. Sellers are no longer required to guess which of the products to focus on, which ones are likely to sell or how much to charge because the assistant provides

predictive insights.

Feature	Single-Tenant SaaS	Multi-Tenant SaaS
Architecture	Each tenant has a dedicated software instance	All tenants share a single instance with logical separation
Cost	Higher cost due to isolated infrastructure	Lower cost as resources are pooled and shared
Maintenance	Updates must be applied separately per tenant	Centralized updates applied once for all tenants
Scalability	Limited, requires scaling per tenant	Highly scalable with dynamic resource allocation
Customization	High customization possible for each tenant	Configurable but with controlled customization
Security	Strong isolation at the instance level	Strong logical isolation; enhanced through role-based access
Resource Utilization	Less efficient due to underutilized dedicated resources	Optimized utilization through sharing and load balancing

Table (i): Single-Tenant SaaS vs Multi-Tenant-SaaS

1.3 CHALLENGES PRESENT IN LEGACY SYSTEMS

The positive outcomes of digitized commerce cannot be ignored, however, many businesses continue to rely on legacy systems, which were never built to meet the agility and speed of the modern marketplace. Such systems are usually monolithic in design, that is, systems that are highly complex, rigid and resistant to any form of change. The sheer cost of maintaining these systems is not justifiable given the manual time and resources that are economically inefficient and is spent on updates, modifications and any form of patches. With respect to sellers, the net effect is high operational costs, given that many businesses are siloed in their systems along with duplicated resources to keep their businesses operational.

Aside from pricing, customization is a major drawback with respect to legacy systems. Sellers are usually restricted with respect to the design and set templates, keeping their business identity highly constrained and their goals very archaic. With respect to the present scenario, the lack of such imaginative capability, in a world where digital competition is highly pervasive and loyalty is dependent only on differentiation personalization, is highly alarming.

Scaling the technology is growth making the problem more challenging. As the number of sellers on the platform and the product catalog grow disproportionately, existing systems face unprecedented challenge. The architecture of such systems was designed to enable growth on a gradual basis. This, in turn, means such businesses tend to miss multiple growth opportunities, not because such opportunities cease to exist, but because technology growth cannot keep pace with business opportunities. In such times that growth prospects remain promising, businesses are able to use the technology to its fullest, the coolant technology can growth and prosper. Otherwise, the falling options would technology growth opportunities.

The lack of supportive systems that can provide intelligent and automated analytics is a problem in its own right. It is more common to find systems that lack advanced features of analytics or predictive capabilities. Because of this gap, sellers are unable to take full advantage of systems that are intended to help sell products by making predictions on optimistic. This is also the reason more businesses are forced to rely on guess work instead of making adequately calculated predictions in order to lessen risk and enable better strategic planning. This challenge is intensified due to excessive integration problem: legacy systems are integration unbundled by design and, as a consequence, do not take advantage of current technologies. Such systems, therefore, are unable to utilize techniques such as artificial intelligence or newer communication methods, such as The Model Context Protocol (MCP), which are swiftly becoming crucial to the domain of digital transformation.

Finally, most legacy platforms function in isolation, offering no real opportunity for collaboration between sellers. In today's connected economy, this is a missed opportunity. A truly modern marketplace should allow sellers not only to showcase and sell their products but also to source from one another, creating partnerships and strengthening the ecosystem. By denying this possibility, outdated platforms inadvertently restrict growth and innovation.

The combination of these factors shows a clear lack of relevance to today's world that legacy systems provide. They are wholly unaligned to the current challenges of the digital economy and are thus insufficient. Businesses must invert their priorities and adopt AI-powered, boundary-less SaaS frameworks. These systems allow higher levels of customization, reduced operating costs, and the ability to scale seamless on a secure, shared infrastructure. In addition, they provide intelligent insights and predictive analytics, which empower sellers to make more informed decisions.

Challenge	Description	Impact
Scalability Limitations	Legacy systems struggle to handle increasing transaction volumes	Performance bottlenecks, lost sales
Maintenance Overhead	Requires frequent manual patches and updates	High operational costs
Integration Issues	Poor compatibility with modern APIs and third-party services	Reduced system interoperability
Security Risks	Outdated security mechanisms vulnerable to attacks	Data breaches, compliance failures
User Experience	Outdated interfaces with poor personalization	Customer dissatisfaction
High Infrastructure Costs	Dedicated servers and hardware requirements	Increased CAPEX and OPEX

Table(ii): Challenges of Legacy E-Commerce Systems

1.4 PROJECT STATEMENT

The objective of this project is to create and deploy a multi-tenant SaaS platform which enables an unlimited number of sellers to build, control, and customize their online stores in a safe, shared environment. Rather than adhere to cookie-cutter designs, sellers can create one-of-a-kind storefronts incorporating their proprietary banners, themes, and layouts to express their brand. Within a simple and intuitive design, sellers can showcase and list products of their liking, organize categories, and perform routine sales functions.

What sets the system apart is its ability to support collaborative commerce. In addition to selling, sellers have the opportunity to acquire products from other vendors in the same ecosystem. Such a feature creates a marketplace which is both transactional and collaborative, allowing businesses to broaden their catalogues, develop collaborative relationships and enhance the network.

The platform enhances seller productivity through the integration of an AI-powered assistant configured using the Model Context Protocol (MCP). This assistant is much more than a digital chatbot; it is a virtual business counsellor. The assistant extracts salutary business information from the sales data, spot customer behavioural patterns, and analytically forecasts data. The assistant is capable of recommending course of action by indicating which products are likely to be more successful, suggesting price changes and recognizing gaps for new products in the market.

These key features of the platform collectively eliminate the drawbacks of the obsolete legacy systems. Rather than exorbitant costs, restricted personalization and dismal scalability, sellers are provided with a solution that is flexible, scalable and smart.

1.5 OBJECTIVES

To develop a cost-effective SaaS solution capable of supporting multiple sellers simultaneously, we aim to let each seller run their businesses independently in a digital environment. The sellers gain complete control of the online stores and are capable of designing a customizable storefront with unique themes, banners and layouts that resonate with their brand. This will allow each seller to maintain their unique identity while utilizing the infrastructure that the platform has to offer.

Within a single platform, sellers will be able to acquire multiple revenues. This, in turn, facilitates cooperative commerce. Sellers are no longer restricted to the selling side of the ecosystem and can now purchase products from other sellers as well. The platform serves as an automated marketplace that not only enhances the business networks of users, but the users themselves through automated partnerships.

The innovative aim of the project is the incorporation of artificial intelligence as a value-added service. Through the Model Context Protocol (MCP), an intelligent assistant is integrated into the platform to perform the function of a business consultant. This AI assistant scrutinizes historical sales records, discerns patterns, and creates predictive analytics to help sellers make optimal, data-driven decisions. By changing from emotion-driven practices to informed strategies, sellers gain a significant advantage in the management and expansion of their business operations.

These objectives, taken together, portray the project as a pioneering offering that combines the agility of SaaS design with the sophistication of artificial intelligence. It does not only solve

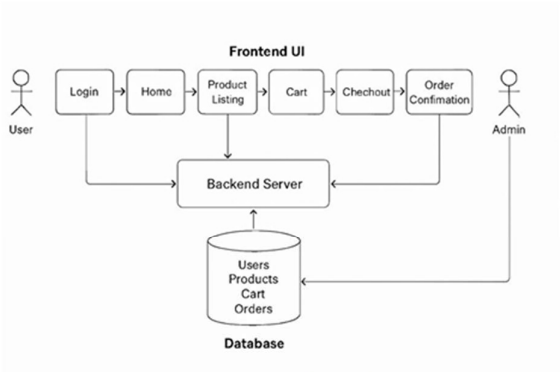
the limitations of legacy systems but offers a robust platform for sustainable growth in the digital economy.

1.6 SCOPE OF THE PROJECT

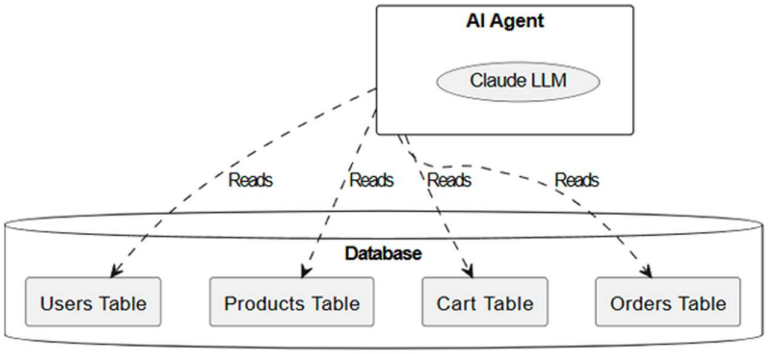
This project aims to design and implement a sophisticated multi-tenant SaaS application that facilitates sellers in starting, configuring, and managing their digital enterprises with ease. The application supports onboarding sellers, personalizing their stores, managing product catalogues and transactions and executing transactions in a shared, secured cloud environment. The platform provides logical data separation within a shared environment to defend privacy and security of all sellers while maintaining the benefits of centralized data management. Under this scenario, the sellers are provided the greatest degree of autonomy to develop their online performance. They can create store themes and banners, organize and sell their products, and manage company sales. The platform also facilitates collaborative marketplace content so that sellers can sell and purchase from each other, fostering inter-business activities and promoting a vibrant ecosystem.

One of the most important features in the scope is the inclusion of an AI-powered assistant based on the Model Context Protocol (MCP). The assistant is designed to act as an intelligence augmentation application by providing real-time analyses of transactions, sales histories, and actionable predictions to guide future strategies. This inclusion greatly enhances the system beyond traditional e-commerce platforms by embedding intelligence directly into business workflows.

1st dataflow diagrams



Claude LLM and Database Interaction



fig(i) Frontend UI Components

fig(ii) Claude LLM and Database Interaction

The dataflow of the system highlights how both users and administrators interact with the platform while also showing how the integration of an AI agent enhances the overall functionality. The process begins with the user, who accesses the application through the frontend interface. From here, the user can log in, browse the home page, view product listings, add items to the cart, proceed to checkout, and finally receive order confirmation. Each of these actions is supported by the backend server, which acts as the central controller of operations. Whenever a user performs an action, the backend server retrieves or stores the necessary information from the database, which holds structured tables containing data about users, products, carts and orders. The administrator also plays a key role in this process, managing the same system through the backend server. Admins are responsible for maintaining product details, managing orders, and ensuring smooth operation of the platform, which makes the system equally effective for both customer-facing and administrative needs.

In addition to this user–admin interaction, the illustration shows the addition of AI agent, Claude LLM, who augments the system's intelligence. The AI agent communicates directly with the database by reading the users, products, cart, and orders tables. This access enables the AI to study and understand pertinent patterns and provide relevant insights. For instance, the AI can automate product recommendations for users, forecast sales for a given period using historical data, and guide admins to make data-driven decisions. The interaction of Claude LLM with the database enhances the system's intelligence and makes the platform operationally functional and able to provide intelligent, anticipatory services.

Adding secure sign-ups and logins, user-friendly designs and the ability to provide scalable backend services alongside the technical intricacies ensures that the system performs reliably and effortlessly for sellers, regardless of their technical prowess. The initial implementation focuses on a web-based application, but the expansion plan deliberately focuses on laying the groundwork for future growth. Further growth areas could include building a mobile application for easier accessibility, adding advanced payment gateways to improve the financial transaction process, and implementing more advanced AI systems to enhance system behaviors such as analytics and tailored recommendations.

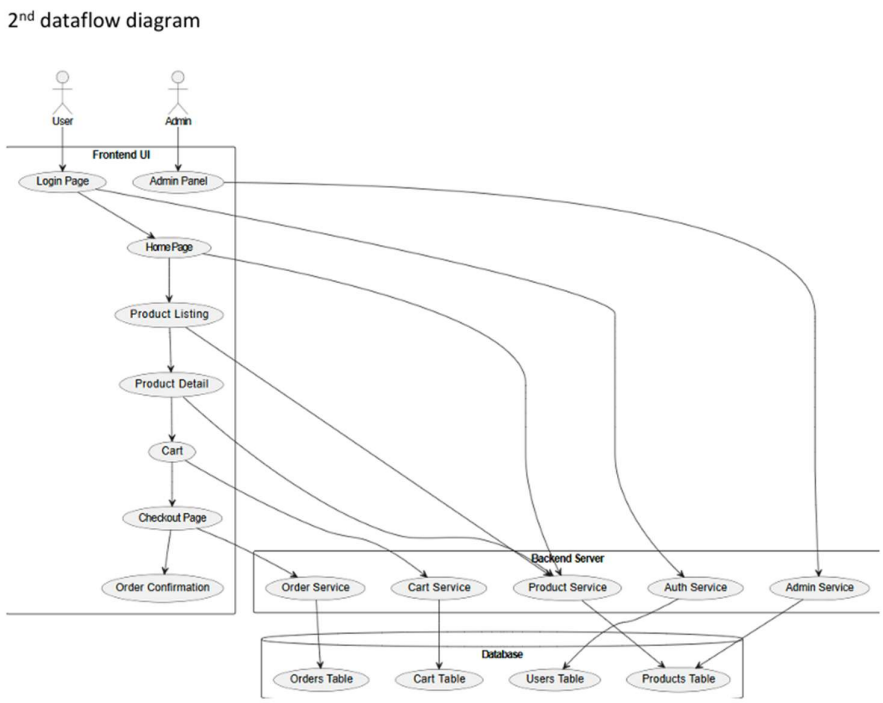
CHAPTER 2

2.1 INTEGRATING MCP IN MULTI-TENANT SAAS FOR DIGITAL COMMERCE

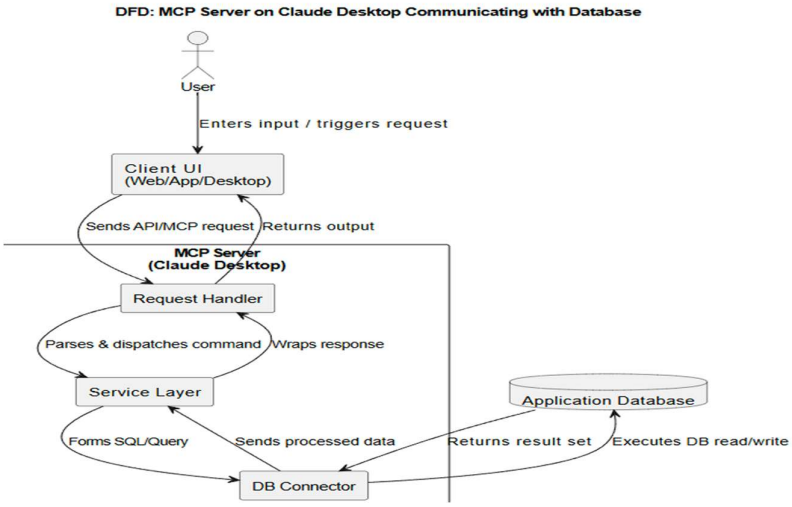
The rise of cloud computing has changed how we develop applications by altering the architecture, distribution, and usage of software applications. Out of the various delivery methods of computing services Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) the SaaS model takes the lead by being the most commonly

used paradigm. SaaS solutions are preferred due to the ease of configuration, low cost, and ease of accessibility. Software applications in the SaaS model are maintained on cloud servers and are accessed online by users, eliminating the need for software installation, specific computing hardware and overall system maintenance.

The idea of multi-tenancy is one of the most important features of this model. Multi-tenancy refers to single application instance serving a number of organizations or users (tenants), while ensuring their data and the business processes remain functionally segregated. This model saves resources, minimizes operational expenditures and provides rapid business updates, simplified administration and easy scalability. Thus, sellers and businesses need to concern themselves with the alleviation of infrastructure pains.



fig(iii) Communication Frontend UI with Backend server and Database



fig(iv) MCP Server on Desktop Communicating with Database

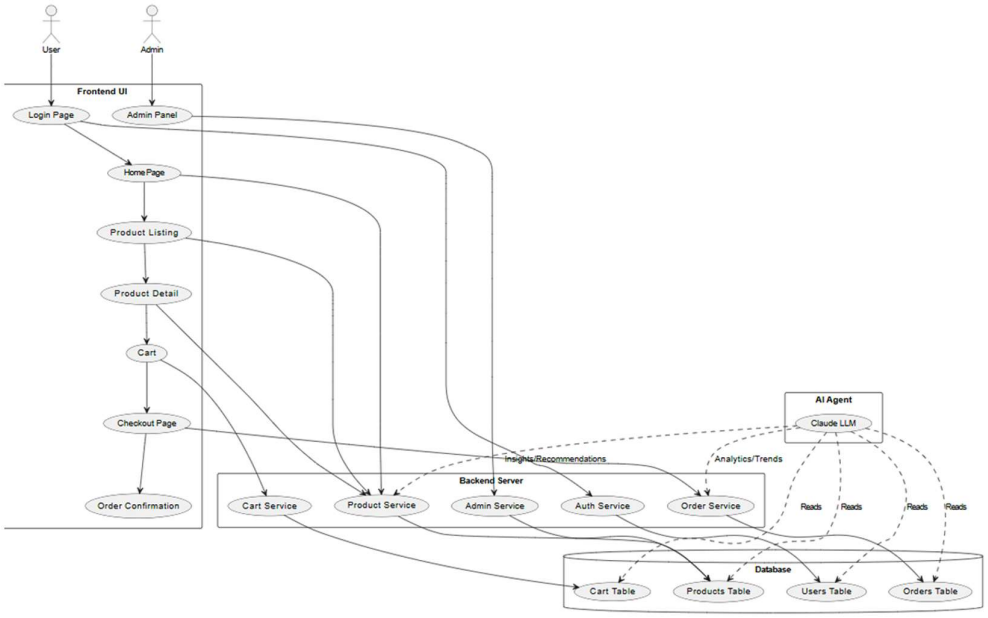
Figure (iii) details how system components interact with the interface and the database. Users and administrators access the application from the client interface; users log in, move to the home screen, view the list of products, examine product details, add products to the shopping cart, go to the checkout page, and confirm the order. Administrators access an admin panel that enables control over products, users and global system functions.

In addition to these interactions, the backend server independently caters to each request and maintains smooth communication with the database and other independent backend modules. Unlike the first diagram, this one focuses on a service-oriented architecture system, where each module's functions are split into independent services. There is an order service that manages order creation and confirmation, a cart service that manages the items selected by the user, a product service with all the product details and listings, an authentication service that manages user logins and other security functions and an admin service with specialized functionalities for and admin user. Each of these services works with its own dedicated database, which includes an orders table, a cart table, a products table and a users table. Each of these services works with a dedicated service database table. This modular approach makes the application secure, maintainable and scalable. Each individual service can be altered, updated, or extended independently without compromising the integrity of the entire system.The fig(iv) describes the communication of the Claude Desktop's MCP server with the application database to increase the system's intelligence .In this process, the user starts by giving the input or by activating a request via the client interface, which can take the form of web, application or desktop interfaces. The client system then initiates an API or MCP request that is directed to the MCP Server which is in charge of the interaction process. Within the MCP Server, the request handler module is the first to access the command, interprets it, and transfers it to the service layer. The service layer is in charge of constructing the corresponding SQL or database command and sending it to the database connector.
The application for which the connector was designed accesses the application database, performs the read or write functionality, and obtains the result set. In turn, the request handler services the request and the structed response, which the service layer returns, is also the response to the client application where the user and result. This was only an example of how the MCP facilitates multi-turn and context aware interfacing of the user interface with the AI

and the database. By the client side, the user interface, the MCP server, the service logic and the connection to the database each has distinctly understood parts to their functionality, which allows for optimal distribution of the gathered data, easily adjustable increased data and enduring delivery of flow.

In addition to these benefits, the embrace of SaaS models has also spawned the ability to embed Artificial Intelligence (AI) within separate applications. AI augmentations within SaaS systems enable sophisticated functionalities, like tailored product recommendations, predictive insights, automated systems and adaptive business strategies. With these, enterprises are able to shift from reactive management to proactive and insightful growth strategies.

In the interest of strengthening the integration of AI constituents to SaaS applications, novel interoperability frameworks like the Model Context Protocol (MCP) have emerged. The MCP Protocol enables context-rich dialogue between the AI assistants and the operating platform, thus granting the AI the ability to offer insights at the level of each tenant. For businesses, particularly sellers within e-commerce marketplaces, integration of SaaS infrastructure with AI through MCP is transformative as it allows the businesses to extract predictive insights from historical sales, customer behavior and market data and powerful trends and dynamics.



fig(v) Communication Frontend UI with Backend server, Database and LLM

After the integration of user interaction, backend services, databases, and AI, the fig(v) captures the essence of the entire system onto a single page, presenting a more integrated and detailed view of the entire system. A user and an administrator are the two dominant components of this interaction center. The user logs in, and accesses the homepage, browses, views detail of a product, adds it to cart, proceeds to the checkout page and finally confirms the order. Simultaneously, the Sys Admin manages the system through an Admin Panel, where he can supervise, update users and products and manage orders.

All of these admin and user actions are executed on a dedicated frontend which communicates directly to a backend server. Instead of a primitive monolithic backend server, here it is service-oriented and divided into several logical, well-defined modules for. The cart service stands by

for to track user-selected items, the product service organizes and controls the display of product listings and related product details, the order service processes and confirms purchases, the authentication service facilitates logging by validating user credentials and controlling user access and the admin service provides management functions that are only available to admins. Each service interacts with a well-defined set of database tables, maintaining the integrity and consistency of user, product, cart and order information.

The database is splitting into users, products, cart and orders tables and describes specific functionality of the application. For the cart services, there is cart table where services populate and remove products. For the orders service, there is cart table from which the service reads and orders table where it writes to. With products table, other services like product service get details. To perform authentication, the users table is accessed. This approach permits independence of each service in order to achieve improvement or scaling.

AI integration with the system via the Claude LLM module enhances the system with new levels of user interaction and system insight. Claude LLM can generate analyses and insights beneficial to both the users and the administrators. Positive user experience Claude LLM can provide users with personalized tips based on their browse and purchase history to make the decision-making process easier. System administrators business decisions can be made with the help of Claude LLM as it captures customer response to products and services and the correlation with the achieved orders.

This evolution marks a major leap in digital commerce. SaaS platforms, when combined with AI intelligence and supported by protocols like MCP, are no longer just operational tools they become strategic ecosystems. They help businesses remain competitive, make smarter decisions, and adapt swiftly in today's fast-paced digital economy.

2.2 SURVEY ON MULTI-TENANT SAAS AND LEGACY SYSTEMS

The studies done so far have indicated that both academia and industry focus on the efficiency and cost savings of multi-tenant SaaS platforms. It is now the focus of research and adoption due to the efficiency, scalability and reduced operational overheads. Multi-tenancy systems, unlike single-tenant or on-premises systems, which required every organization to maintain its own infrastructure, allow multiple tenants to share a single application instance while keeping their data logically separated. Scholars believe that the new infrastructure lowers and simplifies the cost of delivery, updates, and consistency across tenants. The shifts also enhance the overall system manageability.

Most studies being conducted focus on the problems of legacy systems, which still cling to monolithic structures and outmoded technologies. These systems still demand a large amount of capital and technical resources to operate, yet provide little to no flexibility and system integration which the modern enterprise demands. Most upgrades take too long, service integrations are convoluted, and sky does the limit on how far the system can be scaled. Scholars are in consensus that legacy platforms are unsustainable in fast changing digital markets. In contrast, multi-tenant SaaS applications are proven to offer architectural design modularises, flexible API-based integrations, and automated scaling, all of which aid in servicing wide ranging business requirements and high traffic.

Academics have underscored the role of interoperability protocols for promoting seamless relations between SaaS platforms and AI models. The Model Context Protocol (MCP), for

instance, is considered an innovative advancement to enable contextually aware exchanges. AI assistants, through the MCP, come to grasp the distinct business landscape of every tenant and offer specialized, instead of generic, insights and recommendations. This protocol nullifies the distance between the cloud and the intelligent infrastructure, embracing both flexibility and value easement.

Studies that compare older systems to SaaS ecosystems have always shown SaaS to be more cost-effective, more flexible, more secure, and better performing across the board. SaaS systems greatly lower redundancy through shared infrastructure, offer strong data security through logical isolation and enable businesses to adopt further functionalities seamlessly and within no time. Many data support the claim that the SaaS integration with AI improves the underlying competitiveness by offering features such as predictive analytics and intelligent decision support which are absent otherwise.

Furthermore, the impact of adoption of SaaS on business ecosystems has also been studied by some researchers. They indicate that, in addition to isolating productivity benefits, SaaS multi-tenant models encourage cooperation, organizational boundaries interactions, and knowledge diffusion. For example, as a logical development of SaaS platforms, collaborative marketplaces, in which firms sell and procure each other's products and services, have emerged. Such a development directly responds to the necessity for enterprises to function within connected ecosystems as opposed to isolation.

CHAPTER 3

3.1 SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

The platform is built on a modern technology stack designed for scalability, performance and maintainability.

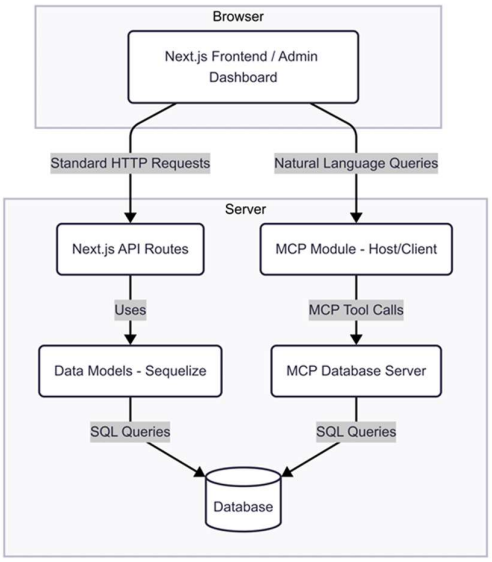
Framework	Next.js 14
Language	JavaScript
Frontend	React.js for building interactive user interfaces.
Backend	Node.js runtime provided by Next.js API Routes.
Database	MySQL, a reliable relational database for structured data storage.
ORM	Sequelize, a promise-based Node.js ORM for managing database interactions and modelling data.
Authentication	JSON Web Tokens (JWT) for securing API endpoints and managing user sessions.

Styling	Standard CSS with CSS Modules for component-level styling.

Table (iii): Technology Stack

The overall architecture follows a client-server model, where the Next.js frontend communicates with its own backend API routes. This integrated approach simplifies development and deployment. The multi-tenancy is achieved at the data level, where every piece of data (products, orders, etc.) is associated with a specific sellerId to ensure logical isolation.

Component Diagram:



Fig(vi) Component Diagram

The system handles both standard application requests and inclined natural language interactions through an integrated workflow of frontend, backend, and database components. The component diagram illustrates this system architecture. Within that hierarchical structure, the browser serves the user interface that together with the Next.js frontend administers the dashboard. Both users and administrators first access this level since it's the direct interface in the system.Consequently, the activities of navigating the system, completing, and even submitting forms is seamless as in every case natural language is provided as an interface.

Two kinds of interactions are registered by the server. The first type includes standard HTTP requests such as browsing the web, filling out forms, and retrieving information. These requests are set up through API routes in Next.js, as is common in web app development. The API routes are tightly coupled with the Sequelize datamodels which abstract the underlying database. In this design, the interaction with the database is limited to the four basic operations: inserts, updates, deletes, and retrieves, involving SQL structured and processed through Sequelize. These SQL queries are run on the database which ensures structured and reliable data, as well as smooth interaction between the frontend and the backend.

The second stream of interaction focuses on more complex functionalities concerning the enabled natural language processing capabilities of the system. Users of the system are no longer restricted to making standard requests but can pose questions in their everyday language. Fulfilling both host and client roles, the MCP module captures these queries and utilizes its smart middleware capabilities to identify natural language queries, commands them, and the appropriate calls to the MCP database server. This database server, meant for interfacing MCP modules that operate on prepare-and-process request level tool-based communication, translates requests into the sufficing executory SQL language of the database. After results are retrieved, the MCP module ensures that the responses are unwrapped from the system, formatted into English, and sent back to the frontend.

The database is the system's backbone as well as the underlying common knowledge base for both streams. It holds all the application data such as users, products, transactions, logs etc. and serves as the foundation for both structured API calls and natural language queries. The database can be accessed through API route and data models using Sequelize. The MCP module, in contrast, adds freedom and logic and enables users to conversate with the system, perform analytic queries and bypass the intricate SQL braids and offers users through certain fully automated insights devoid of the users explicit queries.

3.2 FRONTEND IMPLEMENTATION

The frontend is divided into two main parts: the customer-facing storefront and the seller's admin dashboard.

1. Customer Storefront: This is the public-facing part of each seller's store. The dynamic routing capability of Next.js is used to create unique URLs for each seller (e.g., /store/[slug]).

- Homepage (/[slug]): Displays the seller’s customized banners, featured products, and branding.
- Product & Category Pages: Customers can browse products, filter by category, and view detailed product information.
- Shopping Cart: A global state managed by React Context (CartContext) allows users to add, remove, and update quantities of products in their cart seamlessly across the site.

- Checkout Process: A simple form collects customer details to place an order.

Fig(vii) Activity Diagram

The activity diagram illustrates the complete flow of actions a user goes through when interacting with the e-commerce platform, starting from the initial entry point on the homepage to the final stage of order confirmation or error handling. The process begins when the user visits the homepage, which acts as the starting point for navigation. From there, the user enters the product browsing phase, where different items are displayed for exploration.

When the user clicks on a specific product, the system directs them to the product detail page. At this stage, the user can examine the product more closely, view descriptions, and select preferences such as quantity or weight. Once satisfied, the user chooses to add the item to their

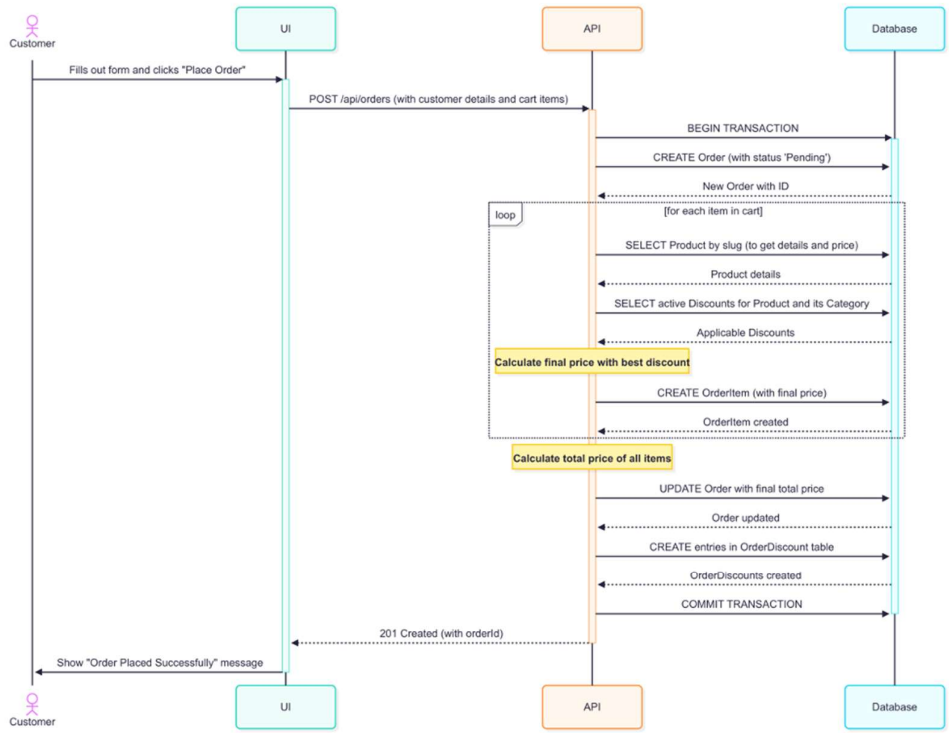
cart. At this decision point, the user faces two possible actions: they can either continue shopping and return to browse more products, or they can move forward to the cart for review.

If the user decides to go to the cart, the system displays the cart page containing all selected items. Here, the user has the flexibility to adjust quantities or remove items. After reviewing the cart and ensuring that all chosen products are correct, the user proceeds to the checkout stage. The checkout process requires the user to fill in necessary details, including shipping information and contact data, which ensures that the order can be processed and delivered accurately.

Once the form is completed, the user places the order by clicking the "**Place Order**" button. This triggers the system to process the order in the backend. The system then evaluates the request and determines whether the order has been successfully processed. If the order is successful, the system displays an "**Order Placed**" confirmation, assuring the user that their purchase has been completed and is on its way. On the other hand, if an error occurs during processing, the system responds by showing an error message, notifying the user of the failure and allowing them to try again or adjust their input.

2. Admin Dashboard: The admin dashboard (/admin) is a protected area where sellers manage their store. Access is controlled via JWT authentication handled by a middleware.

- Dashboard Home: Provides sellers with key statistics like total orders, pending orders, and total products.
- Product Management: A comprehensive interface for performing CRUD (Create, Read, Update, Delete) operations on products, including image uploads.
- Order Management: Sellers can view a list of all orders, see order details, and update the status (e.g., from 'Pending' to 'Shipped').
- Store Customization: This section allows sellers to upload promotional banners and customize the look and feel of their storefront to match their brand identity.
- AI Chat Interface: A dedicated chat window allows sellers to interact with the MCP-powered AI assistant to get business insights.



fig(viii) Sequence diagram for placing order

3.3 BACKEND IMPLEMENTATION

The backend logic is handled by Next.js API Routes located in the src/app/api directory. It is responsible for all business logic, database interactions, and authentication.

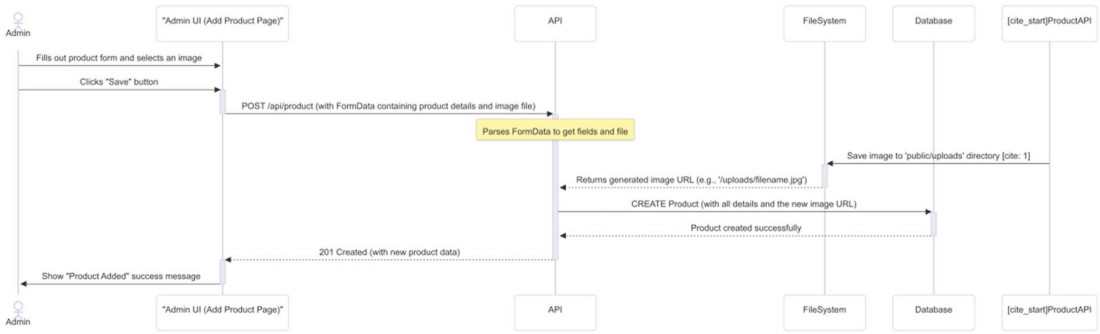
1. **Database and Models:** Sequelize is used to define the database schema through models. Each model file (e.g., Product.js, Order.js, Seller.js) defines the table structure and its associations with other tables. The key to the multi-tenant architecture is the consistent use of a sellerId foreign key in almost every table, which ensures that all database queries are scoped to the currently authenticated seller.

2. **API Endpoints**: The API is designed following RESTful principles. Key endpoints include:

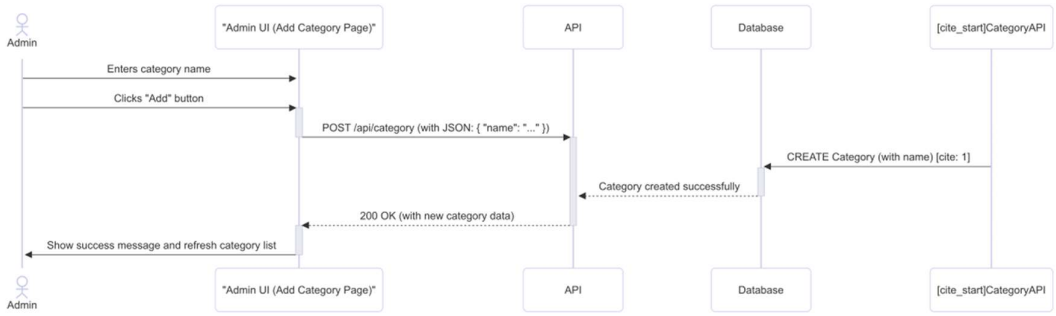
- **/api/sellers/register & /api/login:** Handles seller registration and authentication, issuing a JWT upon successful login.
- **/api/product**: Manages CRUD operations for products. It handles form data, including file uploads for product images.

- /api/orders: Allows customers to create new orders and sellers to retrieve and manage them.
- /api/category: Manages product categories for each seller.
- /api/discounts: Manages the creation and application of discounts.
- /api/admin/*: A collection of protected routes that provide data specifically for the admin dashboard, such as sales statistics and order details.

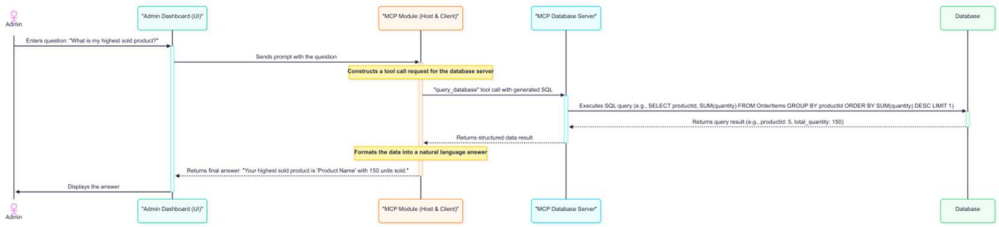
3. Authentication and Security: Security is managed through a middleware (middleware.js) that intercepts requests to protected routes (like /admin). It verifies the JWT stored in the browser's cookies. If the token is valid, the request is allowed to proceed; otherwise, the user is redirected to the login page. This ensures that only authenticated sellers can access their store management dashboard and data. ii. Sequence diagram of adding a product



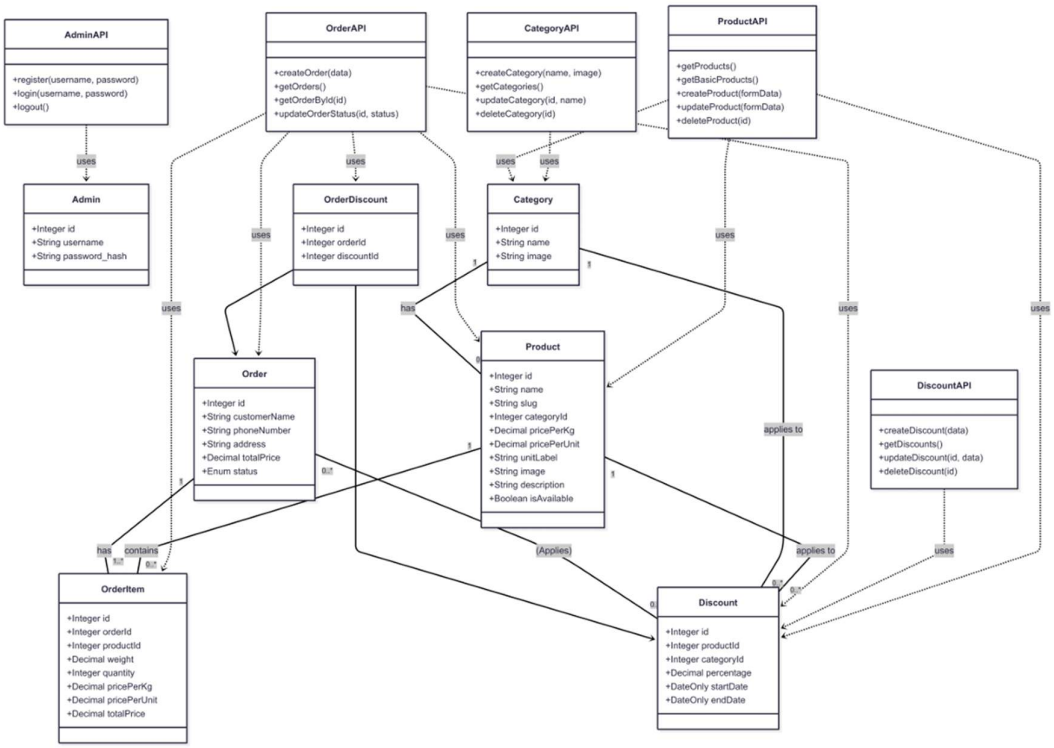
Fig(ix) Sequence diagram of adding product



fig(x) Sequence diagram for adding category



Fig(xi) Sequence diagram for LLM's MCP connection with database and presenting it in seller dashboard



Fig(xii) Class Diagram

The class diagram with APIs provides a detailed representation of the core entities, relationships, and operations that drive the e-commerce platform. At the center of this design lies the **Order**, **Product**, **Category** and **Discount** classes, which form the foundation of the system's business logic. Each of these entities is accompanied by APIs that expose functionality for creating, updating, retrieving, and managing the respective data in a structured manner.

The **Admin** class represents the administrators of the platform, holding attributes such as username and password hash for secure access. Its operations are exposed through the AdminAPI, which enables essential actions such as registering a new admin, logging in with

valid credentials, and logging out. This ensures that system-level tasks such as product or category management remain secure and restricted to authorized users.

The **Order** class captures customer purchase details, including customer name, phone number, address, total price, and status. Each order can contain multiple **OrderItems**, where the specific product, quantity, weight, and calculated price are recorded. The OrderAPI supports order lifecycle management by allowing new orders to be created, existing orders to be retrieved, and order statuses updated. Discounts applied to orders are managed through the **OrderDiscount** class, which links discounts with specific orders, allowing flexible promotional strategies.

The **Product** class represents the items available for purchase, with attributes such as product name, slug, category, unit labels, pricing information, description, and availability status. Products are associated with categories, represented by the **Category** class, which defines logical groupings of products using identifiers, names, and images. To facilitate management, the ProductAPI provides endpoints for creating, updating, retrieving, and deleting product records, while the CategoryAPI handles the creation and organization of product categories.

The **Discount** class introduces promotional features, with attributes such as percentage, start and end dates, and applicability to specific categories or products. This enables sellers to run time-bound offers and apply them dynamically to orders. The DiscountAPI exposes operations to create, retrieve, update, and delete discounts, ensuring seamless integration of promotional campaigns into the sales flow.

Each of these APIs acts as an interface between the underlying data models and external clients, ensuring that system functionality is modular and reusable. The relationships between the entities are clearly defined, with orders containing order items, products belonging to categories, and discounts applying either to products or categories. The diagram also captures how each API interacts with its respective entity to maintain consistency across the platform.

3.4 MODEL CONTEXT PROTOCOL INTEGRATION

The most innovative feature of the platform is the AI assistant, which is powered by the Model Context Protocol. MCP allows the AI to interact with the application's backend in a structured and secure way.

1. MCP Server Endpoint (**/api/mcp/route.js**): An API endpoint is set up to act as an MCP server. This server exposes a predefined set of "tools" or functions that the AI can call. These tools are essentially backend functions that query the database to retrieve specific business intelligence.

2. Predefined Tools: A collection of methods is defined within the MCP server, each designed to answer a specific type of business question. All methods require a sellerId to ensure they only access data belonging to the seller making the request.

Method Name	Purpose	Parameters	Return Data	Business Use Case
get_order_stats	Get daily order statistics	sellerId (required)	{ totalOrdersToday, pendingOrders }	Dashboard overview of current day's performance
get_product_count	Count total products	sellerId (required)	{ totalProducts }	Inventory overview and catalog size
get_category_count	Count total categories	sellerId (required)	{ totalCategories }	Product organization metrics
list_all_categories	Get all category names	sellerId (required)	[{ id, name }]	Category management and navigation
get_products_by_category	Get products in specific category	sellerId, category_name (required)	[productNames]	Category-specific product listings
get_recent_orders	Get recent order details	sellerId, limit (optional, default: 5, max: 50)	[{ id, status, totalAmount, createdAt, items: [{ productName, quantity }] }]	Order tracking and customer service
get_discount_info	Get active discount details	sellerId	[{ id, name, discountPercentage, validFrom, validTo, productName, categoryName }]	Promotional campaign management
get_popular_products	Identify best-selling products	sellerId, days (optional, default: 30, max: 365)	[{ productName, totalSold, price }]	Product performance analysis and marketing
get_business_insights	Comprehensive business intelligence	sellerId	{ insights: [{ type, title, value, description, recommendation }], summary: { totalInsights, criticalAlerts, positiveTrends }, generatedAt }	AI-powered business recommendations and alerts

Table(iv): MCP Server Methods Table

3. Interaction Flow: The process works as follows:

- The seller types a natural language question into the chat interface on their admin dashboard (e.g., "What are my most popular products?").
- The frontend sends this query to a general chat API endpoint (**/api/chat/route.js**).
- This endpoint forwards the query to an external Large Language Model (LLM), instructing it to use the available MCP tools to answer the question.
- The LLM analyzes the query and determines that it needs to call the **get_popular_products** tool. It then makes a request to our **/api/mcp** endpoint with the appropriate tool call.
- Our MCP server executes the **get_popular_products** function, which runs a SQL query to find the best-selling products for that seller.
- The MCP server returns the structured data (e.g., a JSON array of products) to the LLM.
- The LLM formats this data into a user-friendly, natural language sentence (e.g., "Your most popular products are Product A, which sold 50 units, and Product B, which sold 45 units.") and sends it back.
- The final response is displayed to the seller in the chat interface.

This architecture allows for powerful, natural language-based data exploration without exposing the database directly or requiring complex frontend logic.

CHAPTER 4

4.1 TEST INFO

A combination of manual and automated testing was used to verify the correctness of the platform. The testing process was structured around different levels, each focusing on a specific aspect of the system.

Unit Testing:
Unit tests were focused on verifying the smallest individual components of the application in isolation. For the backend, this involved testing individual API route handlers and helper functions to ensure they correctly processed inputs and produced the expected outputs. For example, a unit test for a data validation function would check that it correctly identifies both valid and invalid data formats.

Integration Testing:
Integration tests were designed to verify the interaction between different components of the system. A key focus was testing the integration between the Next.js API routes and the Sequelize models to ensure that data was being correctly passed to and retrieved from the database. For instance, an integration test would simulate an API call to create a product and then query the database directly to verify that the record was created with the correct attributes.

System Testing (End-to-End):
System testing involved testing the application as a whole from an end-user's perspective. This was primarily accomplished using Cypress, an automated end-to-end testing framework. Cypress tests were written to simulate real user workflows, such as a seller registering, adding a product, and a customer subsequently purchasing that product. These tests interact with the application through the browser, clicking buttons, filling out forms, and asserting that the UI updates as expected. This approach is invaluable for catching regressions and ensuring that key user journeys remain functional after new code changes are introduced.

4.2 TEST CASES

To structure the testing process, a series of test cases were designed to cover the critical functionalities of the platform. The results of these tests are summarized in the tables below.

Table 4.1: Test Cases for User Authentication

Test Case ID	Description	Input Data	Expected Result	Actual Result	Status
TC-AUTH-01	Successful seller login	Valid email and password	Redirect to seller dashboard	Redirect to seller dashboard	Pass
TC-AUTH-02	Failed login (invalid password)	Valid email, wrong password	Error message displayed	Error message displayed	Pass
TC-AUTH-03	Failed login (non-existent user)	Unregistered email	Error message displayed	Error message displayed	Pass
TC-AUTH-04	Successful user registration	Valid registration details	Redirect to login page	Redirect to login page	Pass

Successful seller login

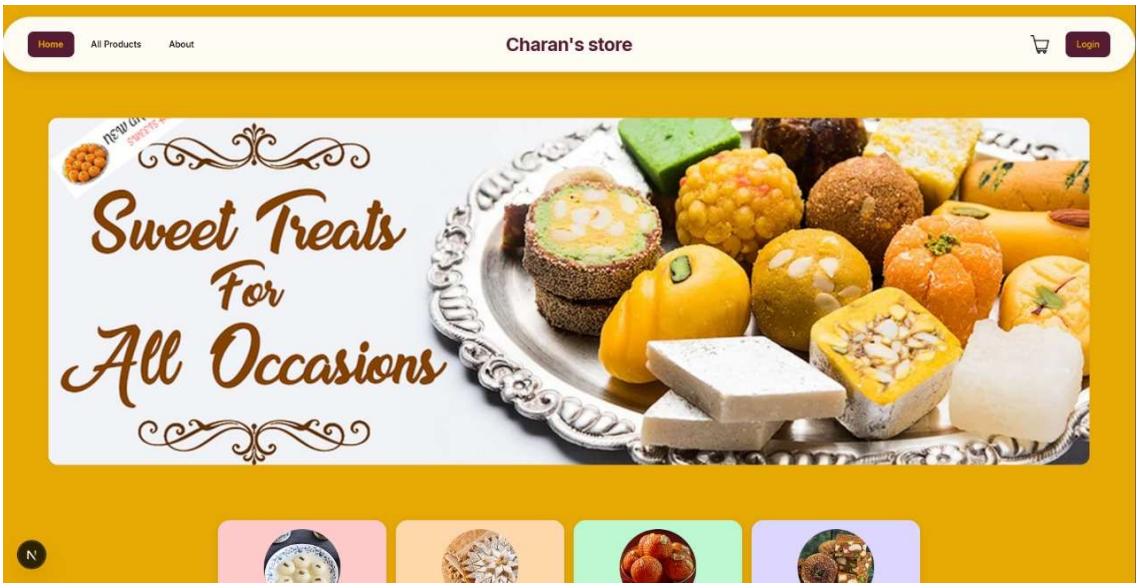


Table 4.1 outlines the test cases for the core authentication module. The scenarios cover successful and failed login attempts for sellers, as well as the new user registration flow. The successful execution of these tests confirms that the authentication logic is robust and provides appropriate feedback to the user under different conditions.

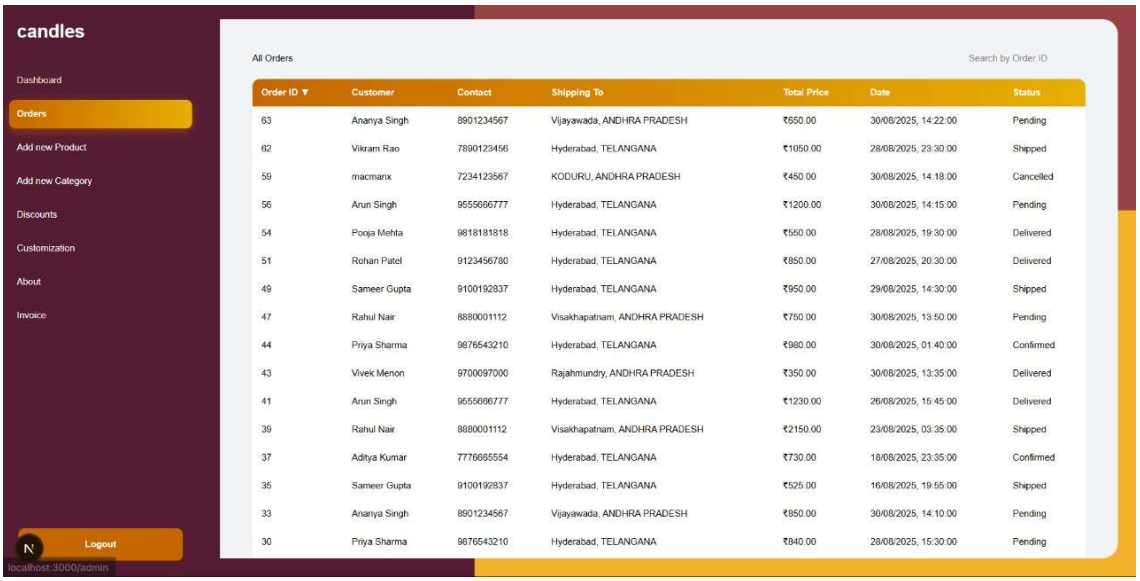
Table 4.2: Test Cases for Product Management

Test Case ID	Description	Input Data	Expected Result	Actual Result	Status
TC-PROD-01	Seller adds a new product	Valid product name, price, stock	Product appears in product list	Product appears in list	Pass
TC-PROD-02	Seller adds product with missing price	Name, stock, no price	Form validation error shown	Validation error shown	Pass
TC-PROD-03	Seller updates an existing product	New price for a product	Product price is updated on storefront	Price is updated	Pass
TC-PROD-04	Seller deletes a product	-	Product is removed from list	Product is removed	Pass

Table 4.2 details the test cases for the product management functionality

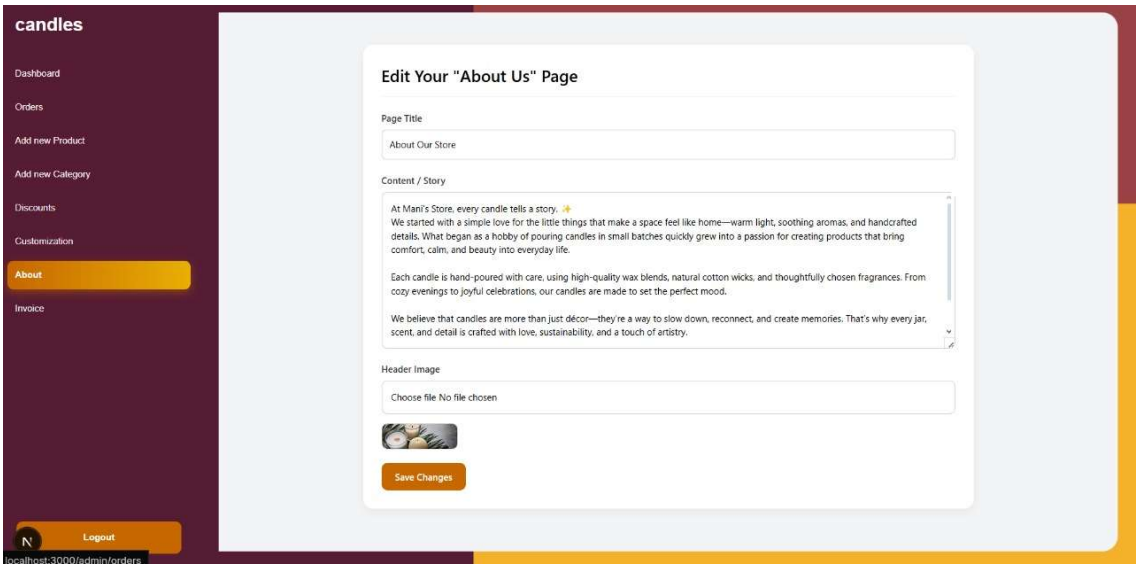
which is a core feature for sellers. These tests validate the complete lifecycle of a product: creation, updating, and deletion. The test cases also include negative scenarios, such as attempting to create a product with incomplete information, to ensure that the system's validation rules are working correctly. The successful results confirm that sellers can reliably manage their product catalog.

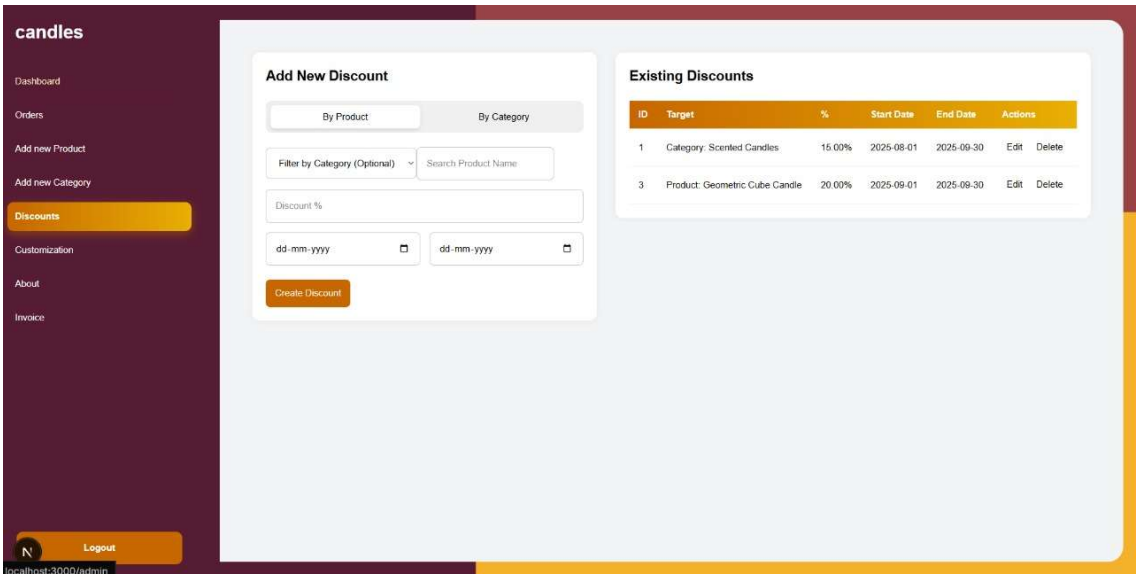
Orders Section



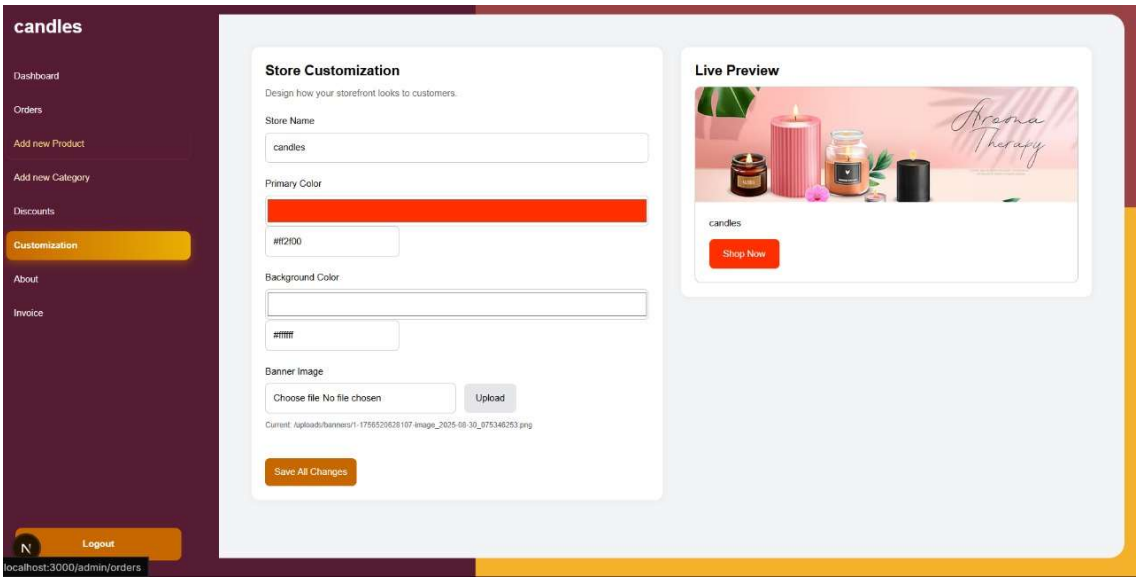
Order ID ▼	Customer	Contact	Shipping To	Total Price	Date	Status
63	Ananya Singh	8901234567	Vijayawada, ANDHRA PRADESH	₹650.00	30/08/2025, 14:22:00	Pending
62	Vikram Rao	7890123456	Hyderabad, TELANGANA	₹1050.00	28/08/2025, 23:30:00	Shipped
59	macmanx	7234123567	KODURU, ANDHRA PRADESH	₹450.00	30/08/2025, 14:18:00	Cancelled
56	Arun Singh	9555666777	Hyderabad, TELANGANA	₹1200.00	30/08/2025, 14:15:00	Pending
54	Pooja Mehta	9818181818	Hyderabad, TELANGANA	₹550.00	28/08/2025, 19:30:00	Delivered
51	Rohan Patel	9123456780	Hyderabad, TELANGANA	₹850.00	27/08/2025, 20:30:00	Delivered
49	Sameer Gupta	9100192837	Hyderabad, TELANGANA	₹950.00	29/08/2025, 14:30:00	Shipped
47	Rahul Nair	8880001112	Visakhapatnam, ANDHRA PRADESH	₹750.00	30/08/2025, 13:50:00	Pending
44	Priya Sharma	9876543210	Hyderabad, TELANGANA	₹980.00	30/08/2025, 01:40:00	Confirmed
43	Vivek Menon	9700097000	Rajahmundry, ANDHRA PRADESH	₹350.00	30/08/2025, 13:35:00	Delivered
41	Arun Singh	9555666777	Hyderabad, TELANGANA	₹1230.00	26/08/2025, 15:45:00	Delivered
39	Rahul Nair	8880001112	Visakhapatnam, ANDHRA PRADESH	₹2150.00	23/08/2025, 03:35:00	Shipped
37	Aditya Kumar	7776665554	Hyderabad, TELANGANA	₹730.00	18/08/2025, 23:35:00	Confirmed
35	Sameer Gupta	9100192837	Hyderabad, TELANGANA	₹525.00	16/08/2025, 19:55:00	Shipped
33	Ananya Singh	8901234567	Vijayawada, ANDHRA PRADESH	₹850.00	30/08/2025, 14:10:00	Pending
30	Priya Sharma	9876543210	Hyderabad, TELANGANA	₹840.00	28/08/2025, 15:30:00	Pending

About Section





Customization



About:

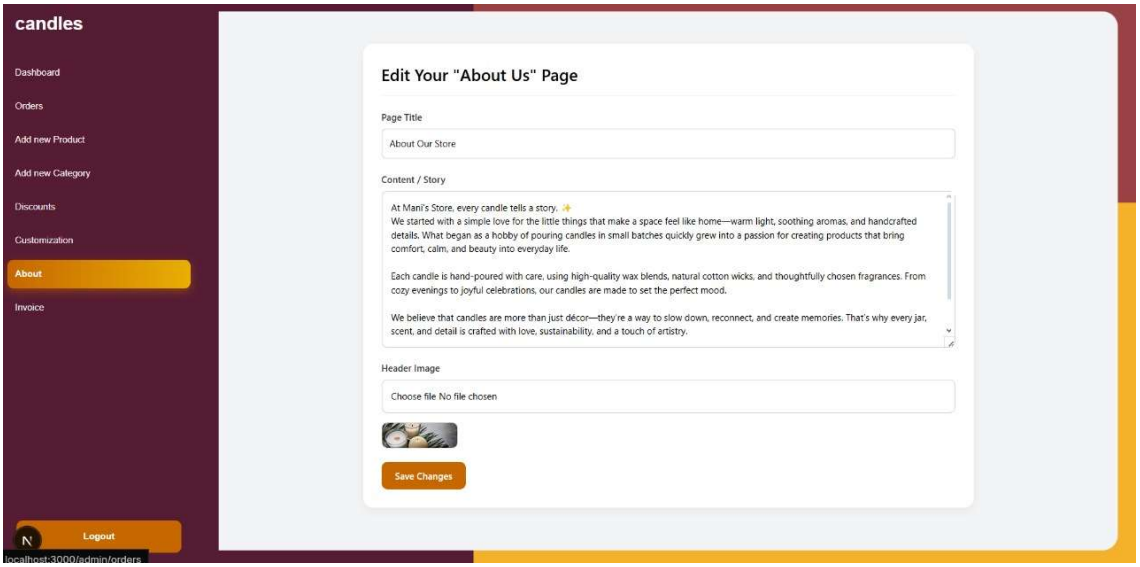
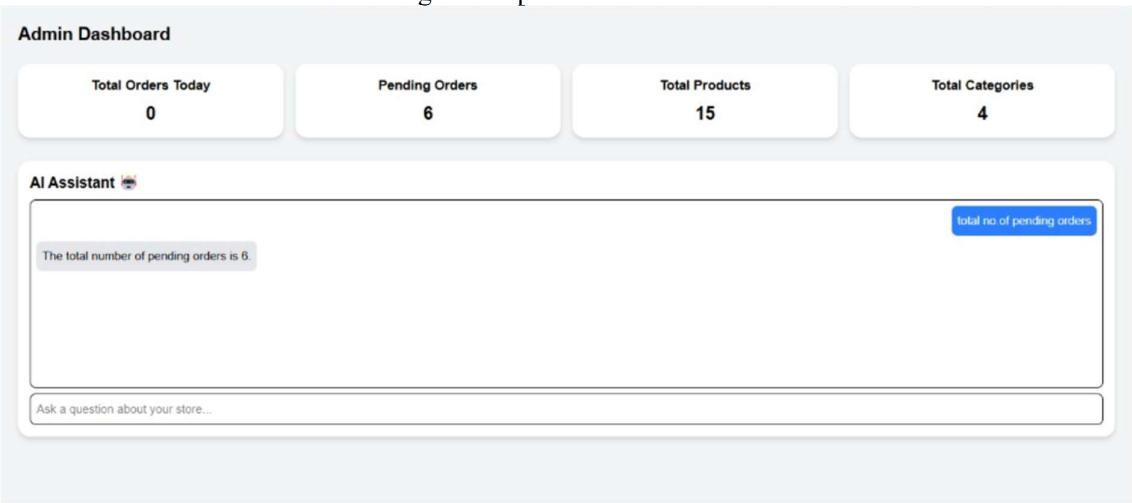


Table 4.3: AI Analytics Module Test Cases

Test Case ID	Description	Input Data (Natural Language Prompt)	Expected Result	Actual Result	Status
TC-MCP-01	Seller queries for count of pending orders	"Total no. of pending orders"	The AI replies with the correct count of orders currently in 'pending' status.	AI replied with correct count.	Pass
TC-MCP-02	Seller requests a categorized list of all products	"List all my products under each category"	The AI replies with a structured list of all products, grouped by their respective categories.	AI replied with correct list.	Pass
TC-MCP-03	Seller queries for top-selling product in a date range	"What was my best selling item last month?"	The AI identifies the product with the highest sales quantity in the previous calendar month and returns its name.	AI replied with correct product.	Pass
TC-MCP-04	Seller makes an ambiguous query	"Show me sales"	The AI asks for clarification (e.g., "For what time period would you like to see the sales data?").	AI asked for clarification.	Pass

Sellers communication with AI through MCP protocol



4.3. TESTING SUMMARY

The comprehensive testing process confirmed that all core modules of the InstantB2C platform are functioning as per the requirements. The authentication system is secure, and sellers are able to manage their products and storefronts effectively. Critically, the AI Analytics Module, tested via the MCP endpoint, successfully interpreted natural language queries and returned accurate, context-specific data. All high-priority test cases passed, validating the platform's readiness for deployment.

< CHAPTER 5

5.1 CONCLUSION & FUTURE WORK

The previous study backing the chapter illustrates the importance of multi-tenant SaaS platforms in the new digital ecosystem. It has been clear from the introductory part that SaaS has transcended the bounds of a low-cost software provisioning model to an innovatively strategic partner of Artificial Intelligence ever since the incorporation of Model Context Protocol (MCP) Adjunct. It has also been learned from the study that though legacy systems in the past supported the operations of a business, they are inadequate in the context of the scalability, flexibility, and intelligence needed in a contemporary, extraordinarily competitive, and data-driven environment.

Research analyzes how multi-tenancy resolves the inefficiencies of legacy infrastructures and moves the inefficient services to eco-friendly infrastructures with smart remuneration services. In addition to these services, the multi-tenancy also drives AI assistants in SaaS applications with predictive analytics and decision scaffolding tested to make the enterprise highly competitive and agile in turbulent environments.

To sum up, the chapter highlights that moving from legacy systems to multi-tenant SaaS with integrated AI is not only a step forward but a must for any organization that wants to grow, streamline operations, and enhance their strategy for a long time to come. This rationale provides context for the remaining chapters that discuss the proposed system's design, deployment, and assessment.

REFERENCES

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., et al. (2010). A view of cloud computing. *Communications of the ACM*, Vol. 53, No. 4, pp. 50–58.

Chong, F. and Carraro, G. (2006). *Architecture Strategies for Catching the Long Tail*. Microsoft Developer Network, pp. 1–26.

OpenAI (2024). *Model Context Protocol (MCP): A Standard for Tooling and AI Integration*. Technical Report, OpenAI Research, San Francisco, USA.

Shevat, A. (2024). *Designing AI Systems with the Model Context Protocol (MCP)*. In: S. Williams (ed.), *Advances in Applied AI Engineering*, O'Reilly Media, pp. 95–120, California.

Johnson, R., Patel, S. and Li, H. (2024). Context-Aware Agent Collaboration through MCP. *Proceedings of the IEEE International Conference on Artificial Intelligence and Systems (ICAIS)*, Singapore, pp. 212–220.

Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, Boston, p. 1272.

Silberschatz, A., Korth, H. F. and Sudarshan, S. (2020). *Database System Concepts*. McGraw-Hill, New York, p. 1376.

Stonebraker, M. and Cattell, R. (2011). 10 rules for scalable performance in 'simple operation' datastores. *Communications of the ACM*, Vol. 54, No. 6, pp. 72–80.